



OPERATIONAL RESEARCH & OPTIMISATION

Dr. Fang He

Email: hef@westminster.ac.uk

Office: N7.118

University of Westminster

Outline

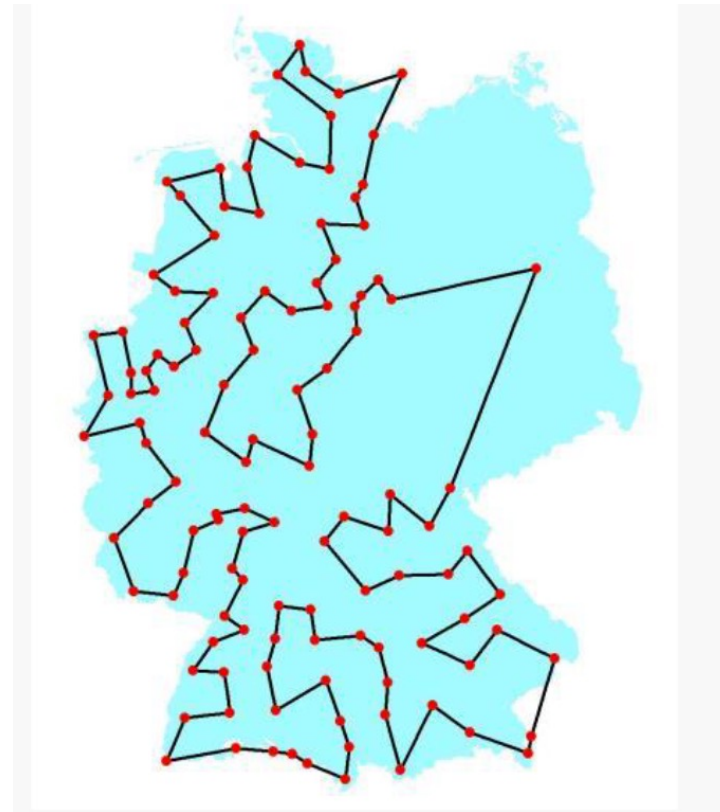
- Introduction to heuristics
- Heuristics illustrated on the traveling salesman problem
- Introduction to Simulated Annealing algorithm
- Design principles for heuristics

Introduction

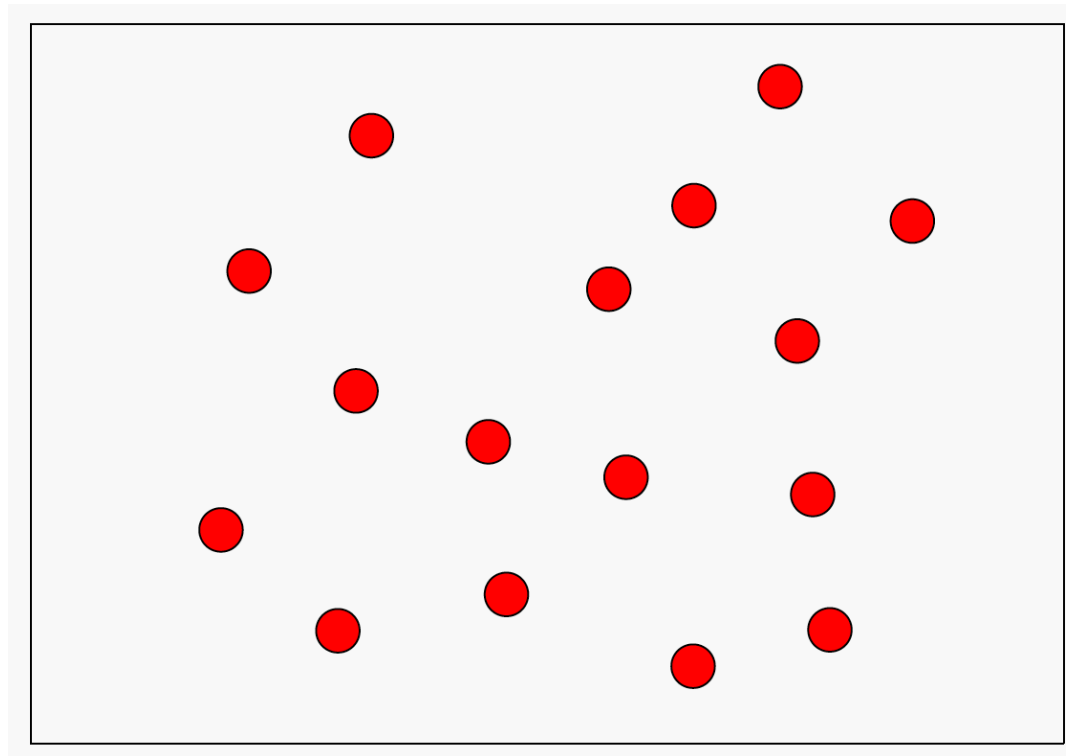
- The real-world problems are usually (if not always) considered hard to be solved because:
 - Problem can not always be represented and solved with a straightforward mathematical approach;
 - A lot of parameters and constraints are involved;
 - The number of possible solution to a problem can be huge;
 - Checking every possible solution, for finding the best one, is time consuming and sometimes not even possible;
 - Good solution need to be found fast.

Traveling Salesperson Problem (TSP)

- Find the shortest distance tour passing through each node of the network exactly once.
- c_{ij} = distance from i to j .
- <http://www.math.uwaterloo.ca/tsp/>



Exercise: Try to find the best tour.



On solving hard problems

- How did you select your tour?
 - it relied on visualization
 - perhaps you took a holistic view
- How can we develop computer heuristics for solving hard problems?

The TSP is a hard problem

- NP-hard (also NP-complete)

Have you seen NP-hardness or NP-completeness before?

1. Yes.
2. No.

The TSP is a hard problem

- There is no known polynomial time algorithm.

Note: In mathematics, a **polynomial** is an expression consisting of variables and coefficients, that involves only the operations of addition, subtraction, multiplication, and positive-integer powers of variables. An example of a polynomial of a single indeterminate x is $x^2 - 4x + 7$. An example with three indeterminates is $x^3 + 2xyz^2 - yz + 1$.

$100 n^{15}$ vs. $n!$

Suppose that we could carry out 1 sextillion steps per second (10^{21}).

- n = number of cities. $(n-1)!$ tours.
- compare time for $100 n^{15}$ steps vs. $n!$ steps.
 - Remark: $100 n^{15}$ steps is NOT practically efficient.

● <u># of cities</u>	<u>$100 n^{15}$ steps</u>	<u>$n!$ steps</u>
$n = 20,$	3.27 seconds	2.4 milliseconds
$n = 25,$	1.5 minutes	4.2 hours
$n = 30$	24 minutes	8,400 years
$n = 35$	4 hours	326 billion years

Heuristic

- The word “heuristic” means “to find” or “to discover” by **trial and error**.
- A heuristic is **a rule of thumb** to guide decision making or to find approximate solutions to optimisation problems.
- Heuristic Search:
 - Heuristic search refers to the techniques with the **aim of finding “good” solutions** for a very hard optimisation and decision problems **within a reasonable amount of computational time**.
 - But, it inability to produce guaranteed optimal solutions. And it is difficult to know, in many cases, how far away from the optimum a heuristic solution might be...

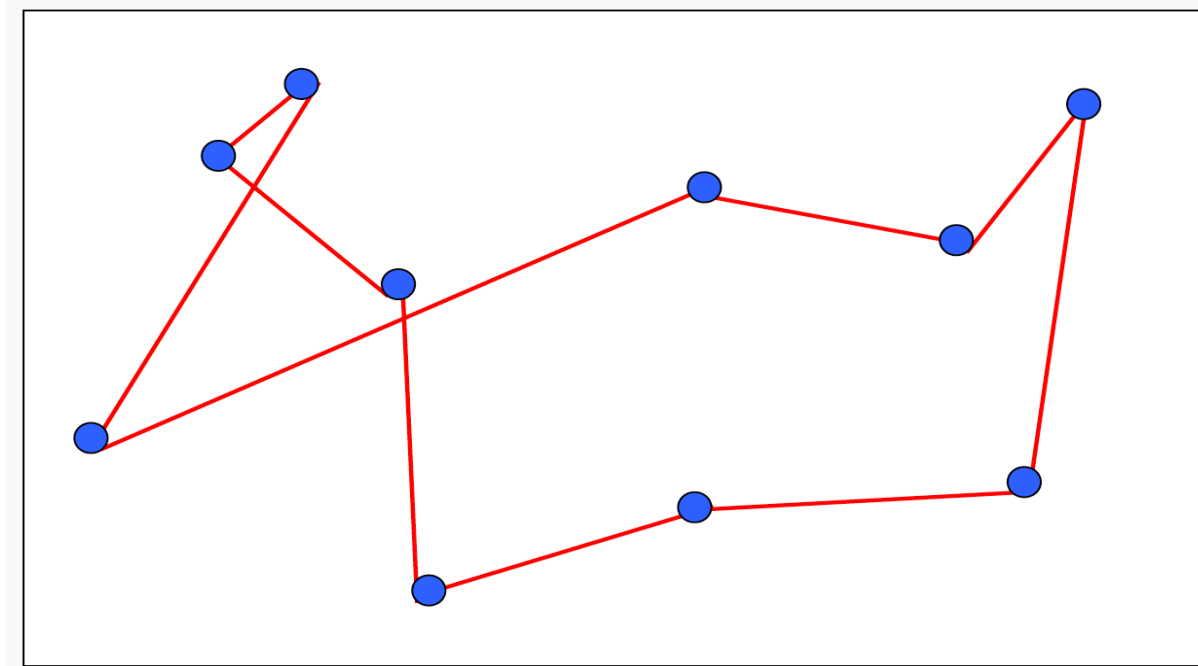
Two types of Heuristics

- **Construction heuristics:** builds a solution from scratch (starting with nothing).
 - Often called “greedy heuristics”. Each step looks good, but it doesn’t look ahead.
- **Improvement heuristics (neighborhood search):** starts with a solution, and then tries to improve the solution, usually by making small changes in the current solution.

Two types of Heuristics

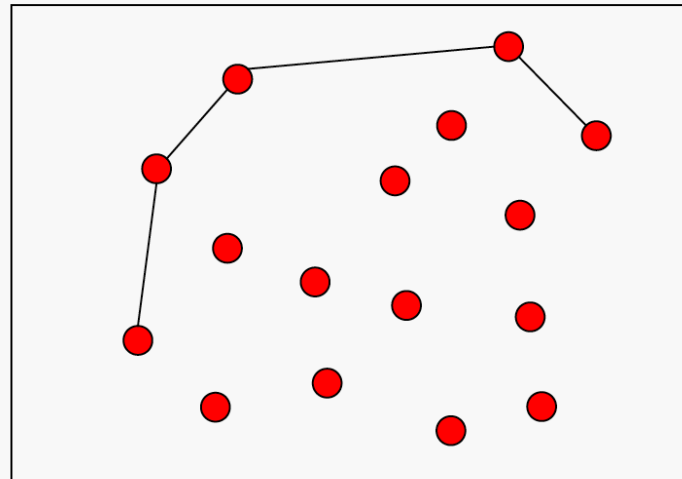
- **Constructive Heuristics:**
 - Simple minded greedy functions: iteratively build a reasonable solution, one element at a time
- **Meta-heuristics:**
 - Single solution based (local search)
 - Simulated Annealing, Tabu Search, Variable Neighbourhood Search, etc.
 - Population based
 - Genetic algorithm, Memetic algorithm, Ant Algorithms, Swarm Intelligence, etc.

An easy construction heuristic: Nearest unvisited neighbour



Can we do better in construction?

- **Class exercise:** try to develop a heuristic in which we add one city at a time, but the next city can be added anywhere in the tour (not just the beginning or the end.)
 - Below is the beginning part of a tour



Introduction to heuristics

- All heuristic search techniques share similar concepts; such as: the search space, feasible/infeasible solutions, neighbourhoods, etc.
- Good understanding of these concepts and how a basic local search works, is a key knowledge necessary to learn and work with heuristics and meta-heuristic algorithms for solving hard real-world optimisation problems.

Introduction to heuristics

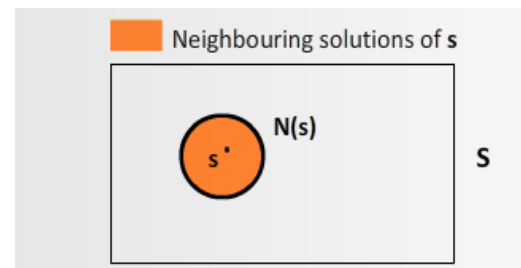
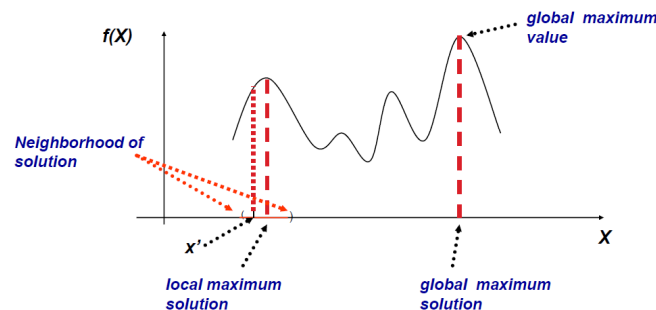
- The set of candidate solutions is **every possible combination of values** that can be assigned to the decision variables of a problem. We refer this as the *search space*.
- We can distinguish candidate solutions, based on whether the constraints are satisfied or not. We refer to them as **feasible** and **infeasible** solutions.

Candidate solutions=search space

Search space = feasible solutions + infeasible solutions

Neighbourhoods

- Neighbourhoods
- Given a search space S and a candidate solution $s \in S$, the neighbourhood $N(s)$ of s can be viewed as a part of the search space that is “close” in some measureable sense to given candidate solution s .
- A neighbourhood is a part of whole search space. A local optimal solution exists within a neighbourhood, we refer it to local optimal.

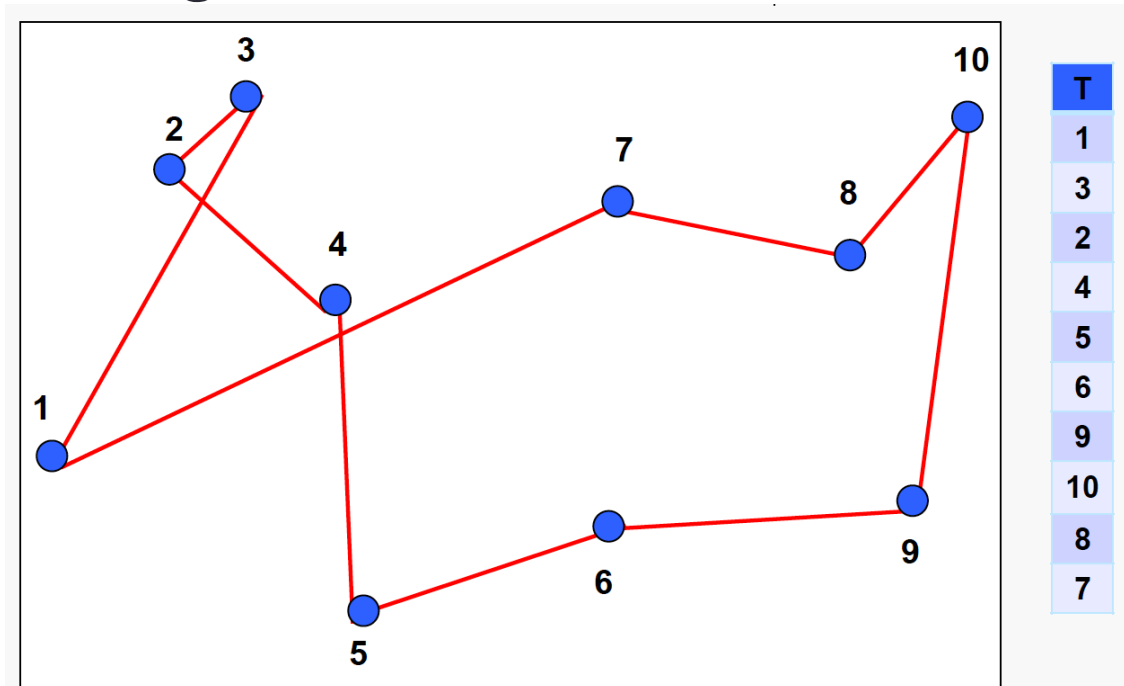


Neighbourhoods

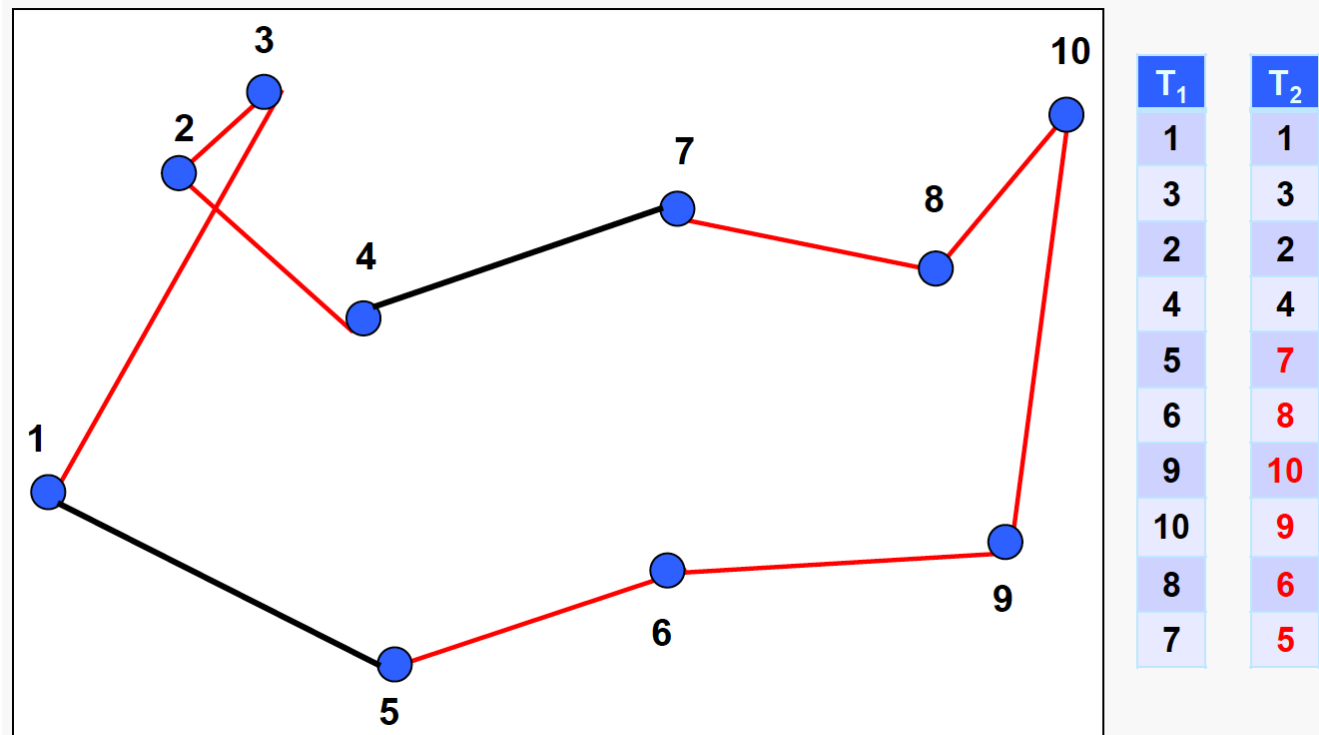
- A neighbourhood function $N(s)$ correspond to the **type of transformation** performed on a given solution s for producing new neighbour solution s' .
- Define the neighbouring solution is **crucial** in the process of **finding high quality local optimal solutions** and it is **strongly problem-specific**.

2 exchanges neighbourhood for TSP

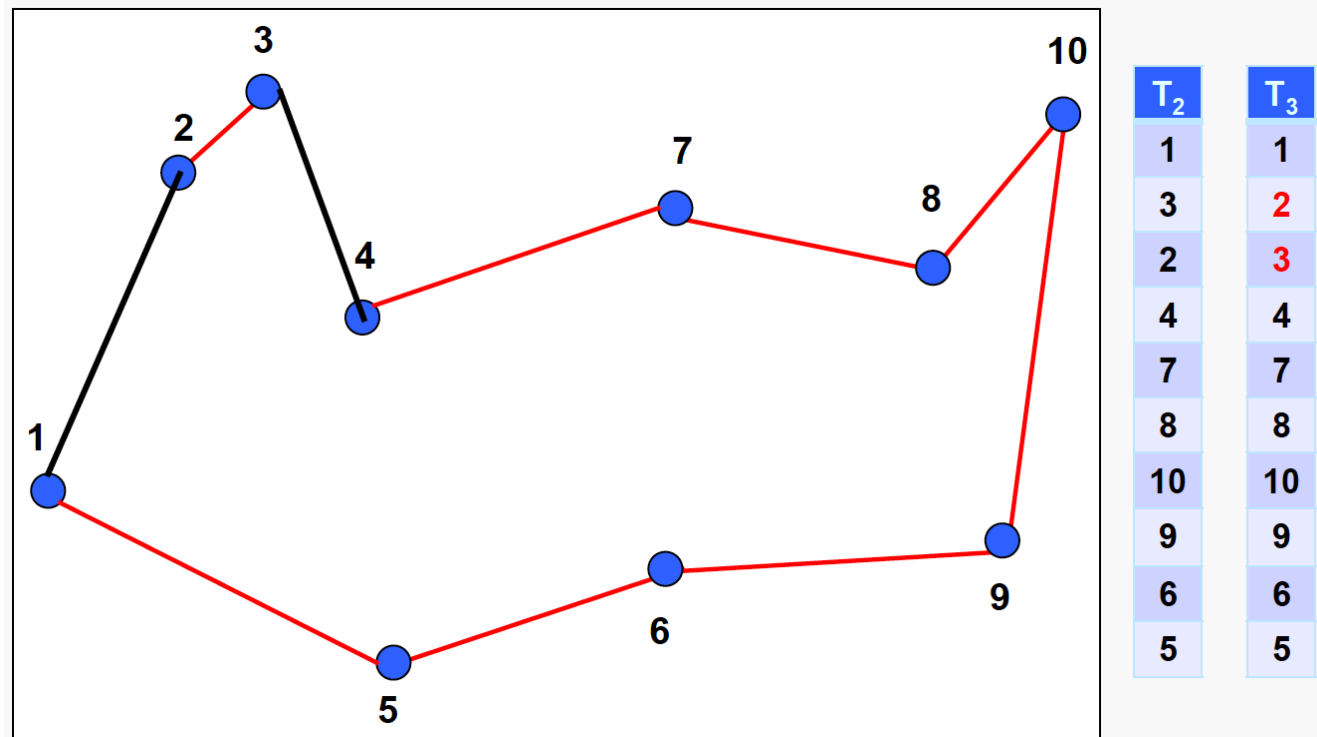
- Neighbourhood is obtained by deleting two edges and adding two edges



After the two exchange

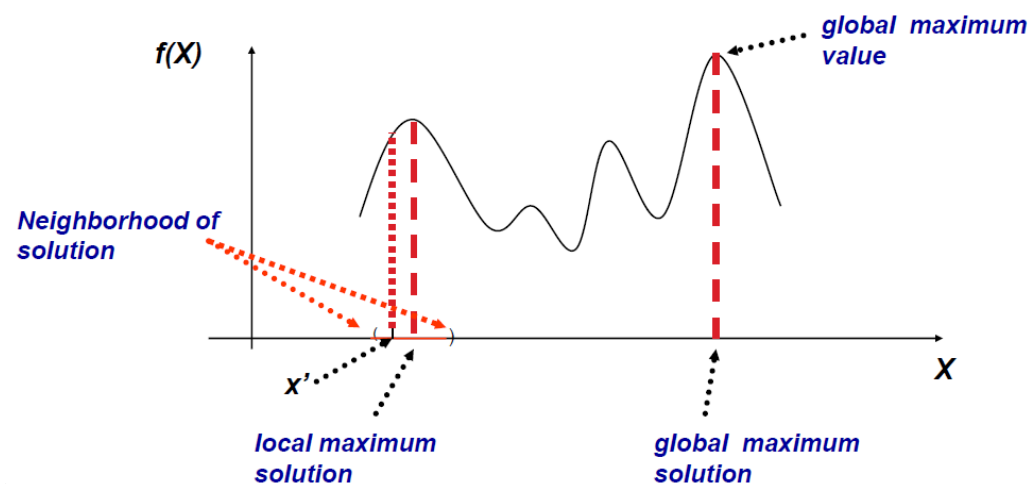


After the two exchange



Local Search

- A basic heuristics search technique that **starts with complete solutions**. And seek to find **better** solutions by making **small local changes**



Local search

- Hill climbing / Steepest Descent
 - “Run uphill / downhill and hope you find the top / bottom of the hills”
- Simulated annealing
 - “Shake it up a lot and then slowly let it settle”
- Tabu search
 - “Don’t look under the same lamp-post twice”
- Variable Neighbourhood Search
 - “Let’s use different transportations i.e. fly / leap / walk, to explore”
- Etc.
- Population based approach
 - Genetic algorithms: “survival of the fittest”
 - Ant algorithms: “wander around a lot and leave a trail”

Local search

- Representation of the solution
 - Solution encoding
- Evaluation function
 - Guide the search
- Neighbourhood function
 - An operator to change (move) a solution to other solutions
- Acceptance criterion
 - First improvement, best improvement, etc

Simulated Annealing

- Physical annealing process: Material is heated and slowly cooled into a uniform structure
- The first SA algorithm: (Metropolis, 1953)
- SA applied to optimisation problems: (Kirkpatrick, 1982)
- Better moves are always accepted
- Worse moves may be accepted, depends on a probability

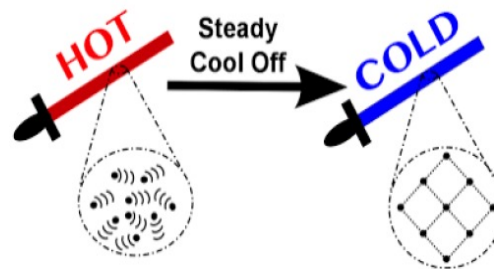
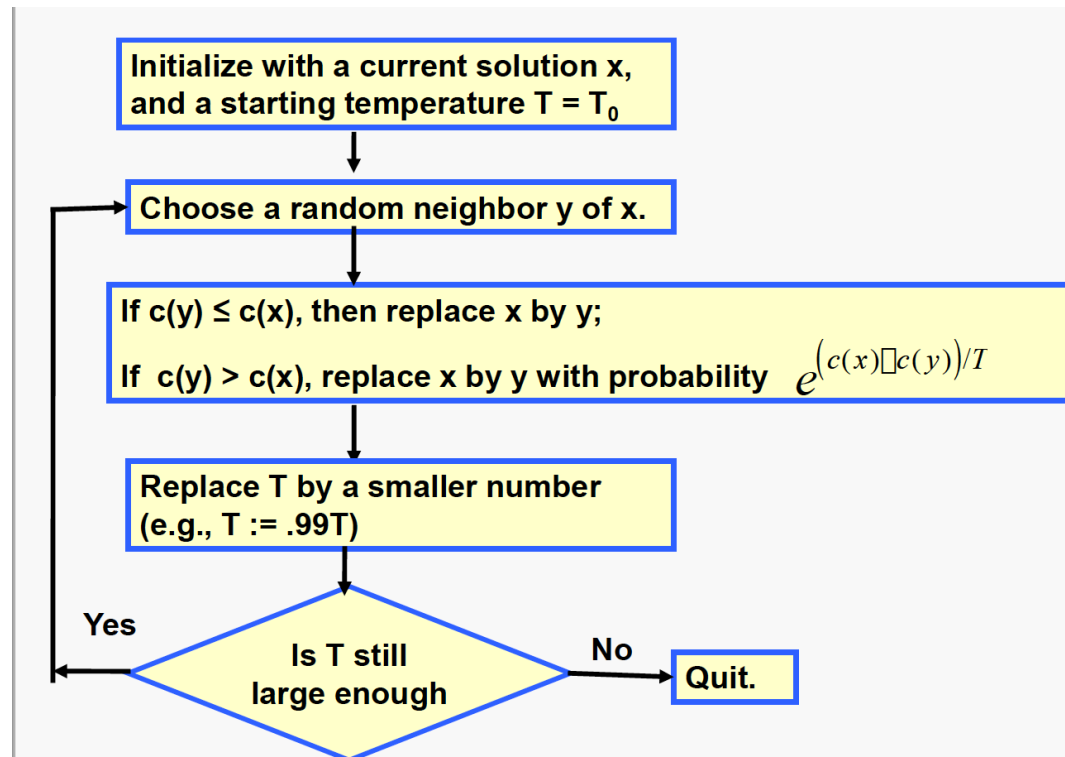


Figure 1: Sword Annealing Analogy to Explain Simulated Annealing (Copyright Jonathan Becker)

Simulated Annealing



Simulated Annealing

Pseudocode:

```
For  $l = 1$  to Iteration do  
     $t = \text{Cooling Schedule}[l]$   
    If  $t = 0$  then return Current  
    Next = random neighbour of Current  
     $c = \text{evaluate}[\text{Next}] - \text{evaluate}[\text{Current}]$   
    if  $c > 0$  then Current = Next  
    else Current = Next with probability  $\exp(-|c|/t)$ 
```

Note: SA: implement greedy search + modified acceptance criteria $\exp(-|c|/t)$
Cooling Schedule is *hidden* in this algorithm: important!

Simulated Annealing

- At temperature t , the probability of accepting a worse solution:

$$P = \exp(-|c|/t) > r$$

c : change in the evaluation function

r : a random number between 0 and 1

t : the current temperature

- The probability of accepting a worse state is a function of
 - the temperature t of the system
 - the change c in the cost function

Simulated Annealing

- The probability of accepting a worse state is a function of
 - the **temperature** t of the system
 - the **change** c in the cost function
- t decreases: the probability of accepting worse moves **decreases**
- $t = 0$: no worse moves are accepted (i.e. greedy search)

Change	Temp	$\exp(-C/T)$	Change	Temp	$\exp(-C/T)$
0.2	0.95	0.810	0.2	0.1	0.13583
0.4	0.95	0.656	0.4	0.1	0.018339
0.6	0.95	0.532	0.6	0.1	0.0024852
0.8	0.95	0.431	0.8	0.1	0.000335

SA-Cooling Schedule

- Starting Temperature
- Final Temperature
- Temperature Decrement
- Iterations at each temperature

SA-Cooling Schedule

- Starting Temperature
 - *hot enough*: to allow almost all neighbourhood (else: greedy search)
 - *not be so hot*: random search for sometime
 - Estimate a suitable starting temperature:
 - Reduce quickly to 60% of worse moves are accepted
 - Use this as the starting temperature
- Final Temperature
 - Usually 0, however in practise, not necessary
 - t is low: accepting a worse move are almost the same as $t = 0$
 - The stopping criteria: either be a suitably low t , or “frozen” at the current t (i.e. no worse moves are being accepted)

SA-Cooling Schedule

- Temperature Decrement
 - Enough iterations at each t , however computationally expensive
 - Compromise:
 - Either: a large number of iterations at a few t 's, or
 - A small number of iterations at many t 's, or
 - A balance between the two
 - Linear: $t = t - x$
 - Geometric: $t = t * \alpha$
 - Experience: $\alpha = (0.8 \text{ and } 0.99)$
 - The higher the value of α , the longer it will take

SA-Cooling Schedule

Acceptance:

$\exp(-|c|/t)$ took about one third of the computation time

- Approximates the exponential (Johnson, 1991)

$$P(c) = 1 - |c|/t$$

- Build a look-up table: values of $|c|/t$
- Speed up the algorithm: about a third with no significant effect on solution quality

Local Search: design and improve

- Evaluation function
 - Calculated at every iteration
 - Often the most expensive part of the algorithm
 - Need be as efficiently as possible
 - Delta / partial evaluation
 - approximate evaluation function, potentially good solutions fully evaluated

Local Search: design and improve

- Evaluation function

Cater for some infeasible solutions using constraints

- Hard Constraints :

- cannot be violated in a feasible solution
- a large weighting: these infeasible solutions have a high cost

- Soft Constraints :

- should, ideally, not be violated but, if they are, the solution is still feasible
- weighted depending importance

- Can be dynamically changed as the algorithm progresses.

- Allows violation of hard constraints to be accepted at the start of the algorithm but rejected later

Local Search: design and improve

- **Initial solution**
 - A random solution: improve
 - A solution that's been heuristically built (e.g. for the TSP problem, start with a greedy search)
- **Hybridisation**
 - Combine two search algorithms
 - The primary search : a population based search
 - A local search is applied to move each individual to a local optimum

Further reading

- Yang, X. S. (2011). Nature-inspired metaheuristic algorithms. Luniver Press.
- Brownlee, J. (2011). Clever Algorithms: Nature-Inspired Programming Recipes. Jason Brownlee.
- El-Ghazali, T. (2009). Metaheuristics: from design to implementation. John Wiley and Sons Inc., Chichester.

- Q&A